DATA SCIENCE INTERNSHIP

TASK 8

Data Cleaning Challenge Using Python

A beginner-friendly data preparation and visualization project

Project area	Data Cleaning and Exploratory Analysis
Main libraries	Python, pandas, NumPy, Matplotlib, seaborn
Dataset	Synthetic student records created in the code

1. Project Overview

Objective: Practice cleaning a messy dataset by handling missing values, duplicate rows, inconsistent data types, and outliers. Then compare the dataset before and after cleaning and save the cleaned data.

Important note: This project generates a small synthetic student dataset for practice. It is not real student information.

2. Requirements

Use Python 3. Google Colab is recommended. Required libraries: pandas, NumPy, Matplotlib, and seaborn.

3. Step-by-Step Instructions

1. Open https://colab.research.google.com/.
2. Create a new notebook.
3. Copy the complete code below into one cell.
4. Run the cell with Shift+Enter.
5. Review the before-and-after summaries and charts.
6. Download the generated CSV files from the Colab file panel if you need to submit them.

4. Complete Python Code

```
# Task 8: Data Cleaning Challenge

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Create a reproducible sample dataset with common data problems
np.random.seed(42)
n = 30

df = pd.DataFrame({
    "StudentID": range(1, n + 1),
    "Name": [f"Student_{i}" for i in range(1, n + 1)],
    "Age": np.random.randint(18, 25, n).astype(object),
    "Attendance": np.random.randint(55, 101, n).astype(float),
    "Marks": np.random.randint(35, 101, n).astype(float),
    "Department": np.random.choice(["CSE", "ECE", "IT", "EEE"], n)
})

# Introduce missing values
df.loc[[2, 7, 14], "Age"] = None
df.loc[[4, 11], "Attendance"] = np.nan
df.loc[[6, 19], "Marks"] = np.nan

# Introduce inconsistent age values
df.loc[0, "Age"] = "twenty"
df.loc[1, "Age"] = "21"

# Introduce a numeric outlier for practice
df.loc[8, "Marks"] = 250

# Add duplicate rows
df = pd.concat([df, df.iloc[[3, 10]]], ignore_index=True)

print("ORIGINAL DATASET")
print("----------------")
print(df.head(10))
print("\\nMissing values before cleaning:")
print(df.isnull().sum())
print("Duplicate rows before cleaning:", df.duplicated().sum())

# Save a copy of the messy dataset
df.to_csv("student_data_messy.csv", index=False)

# 2. Remove duplicate rows
cleaned = df.drop_duplicates().copy()

# 3. Convert Age to numeric; invalid values become missing
cleaned["Age"] = pd.to_numeric(cleaned["Age"], errors="coerce")

# 4. Fill missing numeric values using each column's median
for column in ["Age", "Attendance", "Marks"]:
    cleaned[column] = cleaned[column].fillna(cleaned[column].median())

# 5. Treat marks outside the valid 0-100 range as invalid.
# Replace the invalid value with the median of valid marks.
valid_marks = cleaned["Marks"].between(0, 100)
valid_marks_median = cleaned.loc[valid_marks, "Marks"].median()
cleaned.loc[~valid_marks, "Marks"] = valid_marks_median
```

```
# 6. Standardize text columns
cleaned["Name"] = cleaned["Name"].astype(str).str.strip()
cleaned["Department"] = cleaned["Department"].astype(str).str.strip().str.upper()

# 7. Display the cleaned dataset and checks
print("\\nCLEANED DATASET")
print("---------------")
print(cleaned.head(10))
print("\\nMissing values after cleaning:")
print(cleaned.isnull().sum())
print("Duplicate rows after cleaning:", cleaned.duplicated().sum())
print("Rows before cleaning:", len(df))
print("Rows after cleaning:", len(cleaned))

# 8. Visualize marks before and after cleaning
plt.figure(figsize=(8, 4))
sns.histplot(df["Marks"].dropna(), bins=10, kde=False)
plt.title("Marks Before Cleaning")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("marks_before_cleaning.png", dpi=150)
plt.show()

plt.figure(figsize=(8, 4))
sns.histplot(cleaned["Marks"], bins=10, kde=False)
plt.title("Marks After Cleaning")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("marks_after_cleaning.png", dpi=150)
plt.show()

# 9. Save cleaned data
cleaned.to_csv("student_data_cleaned.csv", index=False)
print("\\nFiles saved:")
print("student_data_messy.csv")
print("student_data_cleaned.csv")
print("marks_before_cleaning.png")
print("marks_after_cleaning.png")
```

5. Code Explanation

NumPy: creates sample values and makes the example repeatable. **pandas:** stores and cleans tabular data. **Matplotlib and seaborn:** create charts to compare marks before and after cleaning.

Missing values: blank Age, Attendance, and Marks values are introduced intentionally. The code fills numeric missing values with the median of each column.

Invalid types: the Age column contains text such as “twenty”. `pd.to_numeric(..., errors='coerce')` converts values that cannot be interpreted as numbers into missing values; those are then filled with the median.

Duplicates: repeated rows are removed using `drop_duplicates()`.

Outlier / invalid value: a Marks value of 250 is added for practice. Since marks are expected to be between 0 and 100, the code replaces values outside that range with the median of valid marks. This is a domain-rule correction, not a general rule for every dataset.

Text standardization: names and department labels are stripped of extra spaces, and department values are converted to uppercase.

6. Expected Output

The notebook prints the first few rows of the original and cleaned datasets, missing-value counts, duplicate counts, and row counts. It also displays two histograms. Exact table values depend on the code and library versions, so use the output from your own run for submission.

```
ORIGINAL DATASET
----------------
[First 10 rows of the generated messy dataset]

Missing values before cleaning:
[Counts for each column]
Duplicate rows before cleaning: 2

CLEANED DATASET
---------------
[First 10 rows after cleaning]

Missing values after cleaning:
[All columns should show 0 missing values]
Duplicate rows after cleaning: 0
Rows before cleaning: 32
Rows after cleaning: 30

Files saved:
student_data_messy.csv
student_data_cleaned.csv
marks_before_cleaning.png
marks_after_cleaning.png
```

7. Files Produced

• **student_data_messy.csv** — original sample data with intentionally introduced issues.
• **student_data_cleaned.csv** — cleaned data.
• **marks_before_cleaning.png** — histogram before cleaning.
• **marks_after_cleaning.png** — histogram after cleaning.

8. Project Report

Title: Data Cleaning Challenge Using Python

Introduction: Real-world datasets can contain missing values, duplicate records, inconsistent types, and invalid entries. Data cleaning improves data quality before analysis or machine learning.

Objective: Identify and correct common data-quality issues in a sample student dataset using Python libraries.

Methodology: A synthetic dataset is generated with missing values, inconsistent age entries, duplicate rows, and a marks value outside the expected range. Duplicate rows are removed, Age is converted to numeric, missing numeric values are filled with medians, invalid marks are corrected using the median of valid marks, and text fields are standardized.

Result: The code saves both messy and cleaned CSV files, prints quality checks, and generates before-and-after marks histograms. The final dataset should have no missing values in the columns created by this example and no duplicate rows.

Conclusion: This project demonstrates a basic and repeatable data-cleaning workflow. Cleaning choices should always be based on the meaning of the data; for real datasets, assumptions such as valid ranges and the best missing-value strategy must be checked with domain knowledge.

9. Troubleshooting

• If imports fail, run `!pip install pandas numpy matplotlib seaborn` in a separate Colab cell.
• If a chart does not appear, check whether the cell finished without an error.
• In Google Colab, use the folder icon to locate and download the CSV and PNG files.
• Keep the messy and cleaned datasets separate so the original data remains available for comparison.

End of Task 8 document